

Testēšanas plāna izstrāde

Sasniedzamie rezultāti:

- Sastāda produkta testēšanas plānu.
- Nosaka testēšanas gadījumus.
- Skaidro testēšanas nozīmi izstrādes procesā.

1. uzdevums. Kāpēc nepieciešama testēšana? Atbildi uz jautājumiem.

1.1. Kas ir programmatūras testēšana?

Programmatūras testēšana ir programmas pārbaude, lai atrastu kļūdas un pārliecinātos, ka funkcijas strādā pareizi un atbilst prasībām.

1.2. Kāpēc testēšana jāveic jau izstrādes laikā?

Testējot izstrādes laikā, kļūdas atklājas agrāk, tās ir lētāk un ātrāk salabot, un mazāka iespēja “salauzt” jau strādājošas funkcijas.

1.3. Nosauc vismaz 3 problēmas, kas var rasties netestētā programmā.

Nekorekti aprēķini

Ievades kļūdas nav noķertas (piemēram: negatīvas vērtības)

UI neatjaunojas (dienas neparādas pēc izvēles)

2. uzdevums. Produkta funkcionalitātes analīze. Nosaki svarīgākās funkcijas savā projektā.

Funkcionalitāte	Lietotāja darbība	Sagaidāmais rezultāts
Cenas aprēķins	Ievada stundas + izvēlas dienas veidu + spiež “Aprēķināt”	Parāda tarifu un kopējo cenu
Validācija	Mēģina iesniegt formu ar tukšiem/nekorektiem laukiem	Parāda kļūdas ziņu, rezultātu nerēķina
Dienas izvēle kalendārā	Atver kalendāru, izvēlas brīvu dienu, saglabā	Galvenajā lapā parādās izvēlēta diena
Aizņemtās dienas	Kalendārā mēģina spiest sarkanu dienu	Diena nav izvēlama, izvēle nemainās

3. uzdevums. Ko nepieciešams testēt? Nosaki galvenos aspektus, kas jātestē Tavā produktā.

Testējamais aspekts	Kāpēc tas jāpārbauda
Funkcionalitāte	lai pārliecinātos, ka visas pogas/soļi strādā no sākuma līdz beigām; testē ar “happy path” (pareiza ievade → pareizs rezultāts).
Lietotāja ievade	lai nepieļautu nekorektu ievadi; testē ar tukšām

	vērtībām, 0, negatīvām, neizvēlētu dienu.
Dizains un izkārtojums	lai lietotājam viss ir saprotams un responsīvs; testē uz šaura ekrāna un dažādos izmēros.
JavaScript darbība	lai nav kļūdu konsolē un DOM atjaunojas; testē kalendāra izvēli, rezultāta attēlošanu, kļūdu paziņojumus.

4. uzdevums. Testēšanas gadījumu plānošana. Izveido testēšanas gadījumus, kas palīdzēs pārbaudīt Tava produkta kvalitāti izstrādes laikā.

Testa Nr	Ko pārbaudīsi	Lietotāja darbība	Sagaidāmais rezultāts
1	Aprēķini	Ievadīt stundas + dienas veids.	2*25=50 EUR
2	Validācija (tukšs dienas veids)	Dienas veids neizvēlēts	Rāda kļūdu
3	Validācija (stunda nav izvēlēta)	Stunda neizvēlēts	Rāda kļūdu
4	Kalendārs (aizņemtas dienas)	Atver kalendāru un mēģina spiest aizņemtu dienu	Saite neļauj
5	Kalendārs -> galvenā lapa	Izvēlas dienu kalendārā	Galvenajā lapā ir redzams izvēlēto dienu.

5. uzdevums. Kļūdainas ievades plānošana. Padomā, kā lietotājs var ievadīt nepareizus datus.

Ievades lauks	Iespējamā kļūdainā ievade	Kā sistēmai jāreaģē
Stundu skaits	Tukšs vai kājoka -3	kļūda jāparādās uzreiz pēc iesniegšanas sistēma rāda kļūdas ziņu
Diena	nav izvēlēta	sistēma rāda kļūdas ziņu ("jāizvēlas diena")

6. uzdevums. Testēšanas plāna izvērtējums. Atbildi uz jautājumiem.

Kā Tavs testēšanas plāns palīdzēs izveidot kvalitatīvāku produktu?

Palīdz atrast kļūdas pirms nodošanas, uzlabo lietojamību, un dod pārliecību, ka aprēķini/validācija strādā vienmēr.

Kad izstrādes procesā Tu veiksi testēšanu?

- | | | | | |
|--------------------------|-----|----------|-----------|----------|
| ☐ | pēc | katras | funkcijas | izveides |
| ☐ | | projekta | | beigās |

☒ regulāri izstrādes laikā

Pamato savu izvēli:

Tā kļūdas tiek atrastas ātrāk un nepazūd laiks lielai labošanu sērijai projekta beigās.

7. uzdevums. Iesniegšana.

1. Saglabā dokumentu kā: `testesanas_plans.pdf`
2. Augšupielādē to savā *GitHub* repozitorijā un ieniedz e-klasē.
3. Veic *commit* ar nosaukumu: `Add testing plan`